

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr.Kesavan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5

9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

*I **S.JAYA SHREE(192524146)**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

S.JAYA SHREE

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Analysis of Requirements

An e-commerce catalogue of 10 million products is subjected to a mixed workload: frequent point lookups by Product_ID, range queries by category/price during browsing, high read-to-write ratio, and periodic bulk inserts during catalogue updates. The chosen organization must minimize average seek cost while keeping insertion overhead manageable.

Recommended Strategy: Hashed Primary Organization + Clustered B+ Tree Secondary Indexes

A hashed file organization (extendible hashing on Product_ID) is recommended as the primary file structure because lookups by Product_ID dominate the workload and hashing gives $O(1)$ average access. To support range queries (category, price), a B+ Tree secondary index is layered on top, since B+ Trees preserve ordering and support efficient range scans that pure hashing cannot.

Cost Analysis

Assume 10,000,000 records, record size ≈ 200 bytes, block size = 4 KB (4096 bytes). Blocking factor $bfr = \lfloor 4096 / 200 \rfloor = 20$ records/block. Number of blocks required $b = \lceil 10,000,000 / 20 \rceil = 500,000$ blocks.

Organization	Avg. Search Cost (blocks)	Insert Cost	Suitability
Heap (unordered)	$b/2 = 250,000$	$O(1)$ – append only	Poor for search-heavy catalogue
Sorted file	$\log_2(b) \approx 19$	$O(b)$ – shift records	Bad for frequent inserts
Hashed (extendible)	$\approx 1-2$	$O(1)$ amortized	Best for Product_ID lookup
Hash + B+Tree (recommended)	1–2 (point), $\log(b)$ (range)	$O(\log B(b))$ per index	Best overall balance

The hybrid approach costs marginally more storage (extra index blocks, $\sim 2-4\%$ overhead) but reduces search cost from 250,000 block accesses (heap) to under 5 for point queries, which is decisive at 10 million rows and justifies the design.

Illustrative Query & Sample Output

```
mysql> CREATE TABLE products (  
-> product_id BIGINT PRIMARY KEY,  
-> category VARCHAR(50),  
-> price DECIMAL(10,2),  
-> name VARCHAR(150)  
-> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> CREATE INDEX idx_category_price ON products(category, price);  
Query OK, 10000000 rows affected (48.21 sec)  
  
mysql> EXPLAIN SELECT * FROM products WHERE product_id = 5032891;  
+----+-----+-----+-----+-----+-----+  
| id | select_type | table | type | possible_keys | key |  
+----+-----+-----+-----+-----+-----+  
| 1 | SIMPLE | products | const | PRIMARY | PRIMARY |  
+----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

Q2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Workload Characteristics

Banking transaction tables grow continuously (append-heavy) and queries typically filter by a specific account_number and a date range (e.g., "transactions for account X in the last 3 months"). This is a classic composite range-query pattern.

Proposed Design

- Primary (clustered) index: a B+ Tree clustering index on account_number, so all transactions for an account are stored physically contiguous — minimizing disk seeks for a single account's history.
- Composite secondary index: a non-clustered B+ Tree index on (account_number, transaction_date) so the date range scan happens directly within the account's leaf-node range, avoiding a full table scan.

- A separate B+ Tree index on transaction_date alone supports bank-wide date-range reports (e.g., daily reconciliation) independent of account.

Why B+ Tree, not Hashing

Hash indexes give $O(1)$ equality lookup but cannot support range predicates such as date BETWEEN, since hashing destroys ordering. B+ Trees keep leaf nodes sorted and linked, so a range query is a single descent to the start key followed by a sequential leaf scan — $O(\log B(n) + k/bfr)$, where k is the number of qualifying rows.

Index Type	Point Query (account)	Range Query (date)	Storage Overhead
Hash index on account_number	$O(1)$	Not supported	Low
B+ Tree on account_number	$O(\log n)$	Partial (needs composite)	Medium
Composite B+ Tree (account_number, date)	$O(\log n)$	$O(\log n + k)$	Medium–High

Illustrative Query & Sample Output

```
mysql> CREATE INDEX idx_acct_date ON transactions(account_number, txn_date);
Query OK, 0 rows affected (12.7 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT txn_id, txn_date, amount
-> FROM transactions
-> WHERE account_number = 'AC10293'
-> AND txn_date BETWEEN '2026-06-01' AND '2026-08-31';
+-----+-----+-----+
| txn_id | txn_date | amount |
+-----+-----+-----+
| 900123 | 2026-06-04 | 1500.00 |
| 900456 | 2026-07-19 | 320.50 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Requirement

Patient records need (a) fast unique lookup by PatientID, and (b) lookup of all patients under a given doctor — a many-to-one access pattern with lower selectivity, since one doctor treats many patients.

Multi-Level Index Design

- Primary index: a sparse, multi-level B+ Tree on PatientID (the ordering/clustering key). Level 0 (leaf) holds actual data block pointers; Levels 1..k hold pointers to the level below — reducing the number of blocks searched from $\log_2(b)$ to $\log_B(b)$, where B is the tree fan-out.
- Secondary index: a dense multi-level B+ Tree on Doctor_Name, since Doctor_Name is not the ordering field of the data file; every distinct doctor name entry points to a bucket/list of matching PatientID pointers (an inverted-list style secondary index).

Why Multi-Level Over Single-Level

A single-level index on 10 million patients with 4-byte pointers and 4KB blocks fits ~500 entries/block, needing ~20,000 index blocks — a single binary search across it still costs $\log_2(20,000) \approx 15$ I/Os if it isn't itself indexed. A multi-level (B+ Tree) index caps this to 2–3 I/Os by keeping upper levels cached in memory, since fan-out is high (hundreds of entries per index block).

Index	Type	Ordering Field?	Structure
PatientID index	Primary, sparse	Yes (clustering key)	Multi-level B+ Tree
Doctor_Name index	Secondary, dense	No (non-clustering)	Multi-level B+ Tree + bucket list

Illustrative Query & Sample Output

```
mysql> CREATE INDEX idx_doctor ON patients(doctor_name);
Query OK, 0 rows affected (3.85 sec)
```

```
mysql> SELECT patient_id, patient_name, doctor_name
-> FROM patients
-> WHERE doctor_name = 'Dr. Meera Nair';
```

```
+-----+-----+-----+
| patient_id | patient_name | doctor_name |
+-----+-----+-----+
| 104521 | R. Subramaniam | Dr. Meera Nair |
| 108832 | K. Varshini | Dr. Meera Nair |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Comparison Framework

Shipment tracking systems have very high insert/delete churn (new shipments created, delivered shipments archived) alongside frequent range queries ("shipments due this week", "in transit between two dates").

Criterion	B-Tree	B+ Tree
Data storage location	Keys + data in internal and leaf nodes	Data only in leaf nodes; internal nodes hold keys only
Range query support	Weak — requires repeated tree traversal	Strong — leaves are linked (doubly linked list)
Fan-out per node	Lower (data takes space in nodes)	Higher (internal nodes hold only keys) → shallower tree
Insert/Delete cost	$O(\log n)$, may rebalance internal nodes with data	$O(\log n)$, rebalancing confined mostly to leaves
Sequential scan	Not directly supported	Very efficient via leaf-chain traversal

Conclusion

B+ Tree is more suitable for this logistics scenario. Its higher fan-out (since internal nodes store only keys, not full records) keeps the tree shallow even at scale, which directly reduces the number of disk I/Os per insert/delete. Just as importantly, the linked-leaf structure makes range queries (date-based shipment tracking) efficient without repeated root-to-leaf traversals — something a plain B-Tree cannot do well. Most production RDBMS engines (MySQL InnoDB, PostgreSQL) use B+ Tree variants for exactly this reason.

Illustrative Query & Sample Output

```
mysql> CREATE TABLE shipments (  
->   shipment_id INT PRIMARY KEY,  
->   status VARCHAR(20),  
->   ship_date DATE,  
->   INDEX idx_ship_date (ship_date)  
-> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> DELETE FROM shipments WHERE status = 'DELIVERED' AND ship_date < '2026-01-01';  
Query OK, 48213 rows affected (1.92 sec)  
  
mysql> SELECT COUNT(*) FROM shipments WHERE ship_date BETWEEN '2026-08-01' AND  
'2026-08-07';  
+-----+  
| COUNT(*) |  
+-----+  
|      6120 |  
+-----+  
1 row in set (0.01 sec)
```

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Design

For 5000 employees, choose Employee_ID as the hash key. With a target load factor of 0.7–0.8 and an average of 20 records/block (4KB block, ~200-byte records), the required number of buckets M is:

$$M = \lceil (5000 / 20) / 0.75 \rceil = \lceil 250 / 0.75 \rceil = 334 \text{ buckets.}$$

Hash function: $h(\text{Employee_ID}) = \text{Employee_ID} \bmod 334$, with each bucket able to chain overflow blocks for collisions (separate chaining).

Static Hashing vs. Extendible Hashing

Aspect	Static Hashing	Extendible Hashing
Bucket count	Fixed at creation time	Grows/shrinks dynamically via directory doubling

Aspect	Static Hashing	Extendible Hashing
Handling growth beyond 5000	Requires full re-hash / re-organization	Only affected bucket(s) split — no full re-hash
Overhead	Minimal (no directory)	Extra directory indirection (small memory cost)
Best fit for	Stable, rarely-changing datasets	Datasets expected to grow (e.g., company hiring)

Recommendation

Because an HR system's headcount grows over time (new hires, attrition) but 5000 is a modest and fairly stable size for a single organization, static hashing with a generous initial bucket count (334, computed above) is adequate and simpler to implement/maintain. If the company anticipates rapid multi-site expansion (e.g., merging headcounts from acquisitions), extendible hashing is preferable since it avoids the costly full re-hash that static hashing would require when the table outgrows its fixed bucket count.

Illustrative Query & Sample Output

```
mysql> CREATE TABLE employees (
  ->   employee_id INT PRIMARY KEY,
  ->   emp_name VARCHAR(100),
  ->   department VARCHAR(50)
  -> ) ENGINE=MEMORY;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT employee_id, emp_name FROM employees WHERE employee_id = 4021;
+-----+-----+
| employee_id | emp_name |
+-----+-----+
|          4021 | S. Praveen |
+-----+-----+
1 row in set (0.00 sec)
```

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Access Pattern

Two distinct query shapes exist: (a) "show me flight AI202" — an exact-match point query, and (b) "show me all flights departing between two dates" — a range query. A single index type cannot serve both optimally, so a combined structure is proposed.

Proposed Combined Index Structure

- A hash index on `flight_number` for $O(1)$ point lookups, since flight numbers are discrete, non-ordered identifiers with no benefit from sorted storage.
- A B+ Tree index on `departure_date` (or a composite (`departure_date`, `flight_number`) index) to serve ordered range scans efficiently, with the linked-leaf structure allowing sequential retrieval of all flights in a date window.
- Optionally, a bitmap index on route/origin-destination pairs if the number of distinct routes is low relative to row count — bitmaps are cheap to AND/OR together for combined filters (e.g., route + date range).

Query Type	Best Index	Expected Cost
Point (<code>flight_number = X</code>)	Hash index	$O(1)$ average
Range (date BETWEEN d1 AND d2)	B+ Tree on <code>departure_date</code>	$O(\log n + k)$
Combined (route AND date range)	Bitmap $\cap$ B+ Tree leaf scan	$O(\log n + k)$, low CPU via bit ops

Illustrative Query & Sample Output

```
mysql> CREATE INDEX idx_flight_hash USING HASH ON flights(flight_number);
Query OK, 0 rows affected (0.11 sec)

mysql> CREATE INDEX idx_dep_date ON flights(departure_date);
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT flight_number, departure_date, status
-> FROM flights
-> WHERE departure_date BETWEEN '2026-09-01' AND '2026-09-03';
+-----+-----+-----+
| flight_number | departure_date | status  |
+-----+-----+-----+
| AI202        | 2026-09-01     | On Time |
| 6E118        | 2026-09-02     | Delayed |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

File Organization

Recommend a clustered sequential (sorted) file organization on Department, since students are naturally grouped by department for administrative processing (department-wise reports, faculty advising), and this ordering is stable (departments rarely change once assigned).

Secondary Indexing for CGPA

CGPA is not the file's ordering field, so a dense secondary B+ Tree index is built on CGPA. Because CGPA-range queries (e.g., "CGPA > 8.5 in Computer Science") combine both fields, a composite secondary index on (Department, CGPA) is preferable to two independent single-column indexes — it lets the optimizer use one index to satisfy both the equality filter (Department) and the range filter (CGPA) in a single ordered scan.

Component	Choice	Reason
Primary file organization	Sorted/clustered on Department	Matches dominant administrative access pattern
Secondary index	Composite B+ Tree (Department, CGPA)	Serves equality + range in one index
Alternative single index on CGPA	Not preferred alone	Would require extra filter step per department

Illustrative Query & Sample Output

```
mysql> CREATE INDEX idx_dept_cgpa ON students(department, cgpa);
Query OK, 0 rows affected (0.87 sec)
```

```
mysql> SELECT student_id, student_name, cgpa
-> FROM students
-> WHERE department = 'CSE' AND cgpa >= 8.5
-> ORDER BY cgpa DESC;
```

```
+-----+-----+-----+
| student_id | student_name | cgpa |
+-----+-----+-----+
|      20145 | A. Keerthana | 9.42 |
|      20198 | R. Dinesh    | 8.76 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given Data

Number of records $r = 100,000$; record size $R = 50$ bytes; block size $B = 4096$ bytes (4 KB).

Blocking factor $bfr = \lfloor B / R \rfloor = \lfloor 4096 / 50 \rfloor = 81$ records/block.

Minimum blocks required $b_{\min} = \lceil r / bfr \rceil = \lceil 100,000 / 81 \rceil = 1,235$ blocks.

Utilization by Organization

Organization	Blocks Used	Utilization	Notes
Heap (unordered)	1,235 ($b_{\min}$)	$\approx 100\%$	Packed tightly; no reserved space, but $O(b)$ search
Sorted file	1,235–1,350 (~5–10% free space reserved)	$\approx 90\text{--}95\%$	Reserves slack space per block to reduce shifting cost on insert
Hashed (static, load factor 0.75)	$\lceil 1,235 / 0.75 \rceil \approx 1,647$	$\approx 75\%$	Deliberately under-filled to limit collision/overflow chains

Interpretation

Heap files achieve the highest raw block utilization since records are simply appended with no reserved space, but this comes at the cost of $O(b)$ linear search. Sorted files trade a small utilization loss (5–10% reserved slack) for the ability to binary-search in $O(\log_2 b)$. Hashed files intentionally operate below full capacity (load factor ≈ 0.75) — the ~25% "wasted" space is what keeps collision chains short and preserves near $O(1)$ average access time. The apparent inefficiency in hashing is therefore a deliberate performance trade-off, not a design flaw.

Illustrative Query & Sample Output

```
mysql> SELECT
->   table_name,
->   ROUND(data_length/1024/1024, 2) AS data_mb,
->   ROUND(data_free/1024/1024, 2)  AS free_mb
-> FROM information_schema.tables
-> WHERE table_name = 'employee_heap';
+-----+-----+-----+
| table_name | data_mb | free_mb |
+-----+-----+-----+
| employee_heap | 4.83 | 0.02 |
+-----+-----+-----+
1 row in set (0.02 sec)
```

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Challenge

At 500 million rows, three very different access patterns must be served: (1) "all posts by a user, newest first" — a per-user timeline; (2) "all posts under a hashtag, recent first" — trending/search; (3) occasional analytics scans across the full dataset. A single flat index cannot serve all three efficiently at this scale.

Composite Strategy

- Partition the posts table by $\text{hash}(\text{user_id}) \bmod N$ or by time range (monthly partitions) to keep each partition's B+ Tree shallow and manageable — a form of horizontal fragmentation that reduces the effective n in $O(\log n)$.
- Composite clustered index on $(\text{user_id}, \text{timestamp DESC})$ to serve per-user timelines directly in sorted order without a separate sort step.
- Inverted index on hashtag $\rightarrow$ posting list of $(\text{post_id}, \text{timestamp})$, similar to search-engine indexing, since hashtags are highly repetitive (low cardinality relative to 500M rows) and a B+ Tree on hashtag alone would still need to sort a huge posting list by recency — the inverted list should itself be timestamp-ordered.
- A secondary composite index on $(\text{hashtag}, \text{timestamp DESC})$ for the trending/search path, kept in a denormalized or materialized structure refreshed asynchronously, since exact real-time ordering is less critical than for the personal timeline.

Access Pattern	Structure	Complexity
User timeline	Composite B+ Tree (user_id, timestamp)	$O(\log n + k)$
Hashtag search	Inverted index + (hashtag, timestamp)	$O(\log n + k)$
Full analytics scan	Partitioned heap / columnar scan	$O(n / \text{partitions})$ parallelized

Illustrative Query & Sample Output

```
mysql> CREATE TABLE posts (  
->   post_id BIGINT PRIMARY KEY,  
->   user_id BIGINT,  
->   hashtag VARCHAR(50),  
->   created_at DATETIME,  
->   KEY idx_user_time (user_id, created_at),
```

```

-> KEY idx_tag_time (hashtag, created_at)
-> ) ENGINE=InnoDB
-> PARTITION BY HASH(user_id) PARTITIONS 64;
Query OK, 0 rows affected (2 min 14.09 sec)

mysql> SELECT post_id, created_at FROM posts
      -> WHERE hashtag = '#dbms' ORDER BY created_at DESC LIMIT 5;
+-----+-----+
| post_id | created_at |
+-----+-----+
| 9012345678 | 2026-08-26 21:14:02 |
| 9012344210 | 2026-08-26 18:03:47 |
+-----+-----+
2 rows in set (0.01 sec)

```

Q10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Scenario

A supermarket billing system logs ~1,000,000 transactions/day, each written once at checkout (heavy insert), rarely updated, occasionally deleted (returns/voids), and searched by transaction_id or bill_no for receipts/audits.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$ — append to end of file	$O(n)$ — must shift to maintain order	$O(1)$ average — direct bucket write
Search (by key)	$O(n)$ — linear scan	$O(\log_2 n)$ — binary search	$O(1)$ average — direct hash lookup
Delete (by key)	$O(n)$ — find, then mark/remove	$O(n)$ — find $(\log n) + \text{shift } (n)$	$O(1)$ average — find bucket, remove
Range search	$O(n)$	$O(\log_2 n + k)$ — efficient	Not supported directly

Recommendation for This Scenario

Given that checkout insert volume (1 million/day) dominates the workload and receipt lookups are by an exact transaction_id, a hashed file organization is the best fit: $O(1)$ average insert and $O(1)$ average search directly address the two most frequent operations. A sorted file would be a poor choice here because every insert would cost $O(n)$ to preserve order — unacceptable at 1 million inserts/day. A heap file is fast to insert but makes every receipt lookup an $O(n)$ scan, which does not scale as the daily

log grows. In practice, a hybrid is common: a hashed structure (or hash-organized clustered index) for the current day's live transactions, periodically archived into a sorted/partitioned structure for historical range-based reporting.

Illustrative Query & Sample Output

```
mysql> CREATE TABLE billing (  
->   txn_id BIGINT PRIMARY KEY,  
->   bill_no VARCHAR(20),  
->   amount DECIMAL(10,2),  
->   txn_time DATETIME  
-> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO billing VALUES (10000001, 'BN-88213', 452.50, NOW());  
Query OK, 1 row affected (0.00 sec)  
  
mysql> SELECT * FROM billing WHERE txn_id = 10000001;  
+-----+-----+-----+-----+  
| txn_id | bill_no | amount | txn_time          |  
+-----+-----+-----+-----+  
| 10000001 | BN-88213 | 452.50 | 2026-08-27 10:02:11 |  
+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```